

Projektowanie Oprogramowania Systemów

Etap 4 - Testowanie przygotowanego oprogramowania.

Szymon Stobiński, 195823

Zbyszko Stachecki, 193621

Hubert Wilarski, 188970

1. Testowanie przygotowanego oprogramowania:

W celu przeprowadzenia testów, utworzono plik z funkcjami testowymi.

```
[Fact]
Odwołania: 0
public void Test_Czy_Temperatura_25_Jest_Poprawna()
{
```

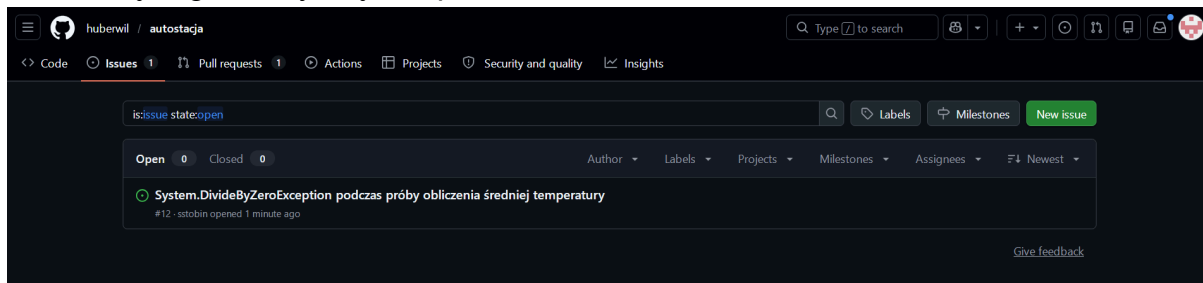
```
[Fact]
Odwołania: 0
public void Test_Czy_Temperatura_Minus_50_Zostanie_Odrzucona()
{
```

```
[Fact]
Odwołania: 0
public void Test_Czy_Srednia_Dla_Braku_Danych_Zwraca_Zero_Bez_Awarii()
{
```

Nie wszystkie testy zakończyły się pomyślnie.

```
Trwa rozpoczynanie wykonywania testu, czekaj...
łączna liczba plików testowych dopasowanych do określonego wzorca: 1.
[xUnit.net 00:00:00.37] WeatherStation.Tests.TemperatureTests.Test_Czy_Srednia_Dla_Braku_Danych_Zwraca_Zero_Bez_Awarii [FAIL]
Niepowodzenie WeatherStation.Tests.TemperatureTests.Test_Czy_Srednia_Dla_Braku_Danych_Zwraca_Zero_Bez_Awarii [1 ms]
Komunikat o błędzie:
System.DivideByZeroException : Attempted to divide by zero.
Ślad stosu:
   at WeatherStation.WeatherData.GetAverageTemperatureLastDays(Int32 days) in C:\Users\Szymon\Desktop\StacjaPogodowa\WeatherStation\Program.cs:line 418
   at WeatherStation.Tests.TemperatureTests.Test_Czy_Srednia_Dla_Braku_Danych_Zwraca_Zero_Bez_Awarii() in C:\Users\Szymon\Desktop\StacjaPogodowa\WeatherStation.Tests\TemperatureTests.cs:line 40
   at System.RuntimeMethodHandle.InvokeMethod(Object target, Void** arguments, Signature sig, Boolean isConstructor)
   at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj, BindingFlags invokeAttr)
Niepowodzenie! – niepowodzenie: 1, powodzenie: 2, pominięto: 0, łącznie: 3, czas trwania: 18 ms - WeatherStation.Tests.dll (net8.0)
```

Został więc zgłoszony błąd na platformie GitHub.



Błąd został naprawiony przez edycję pliku main.cs, dodając odpowiedni warunek.

```

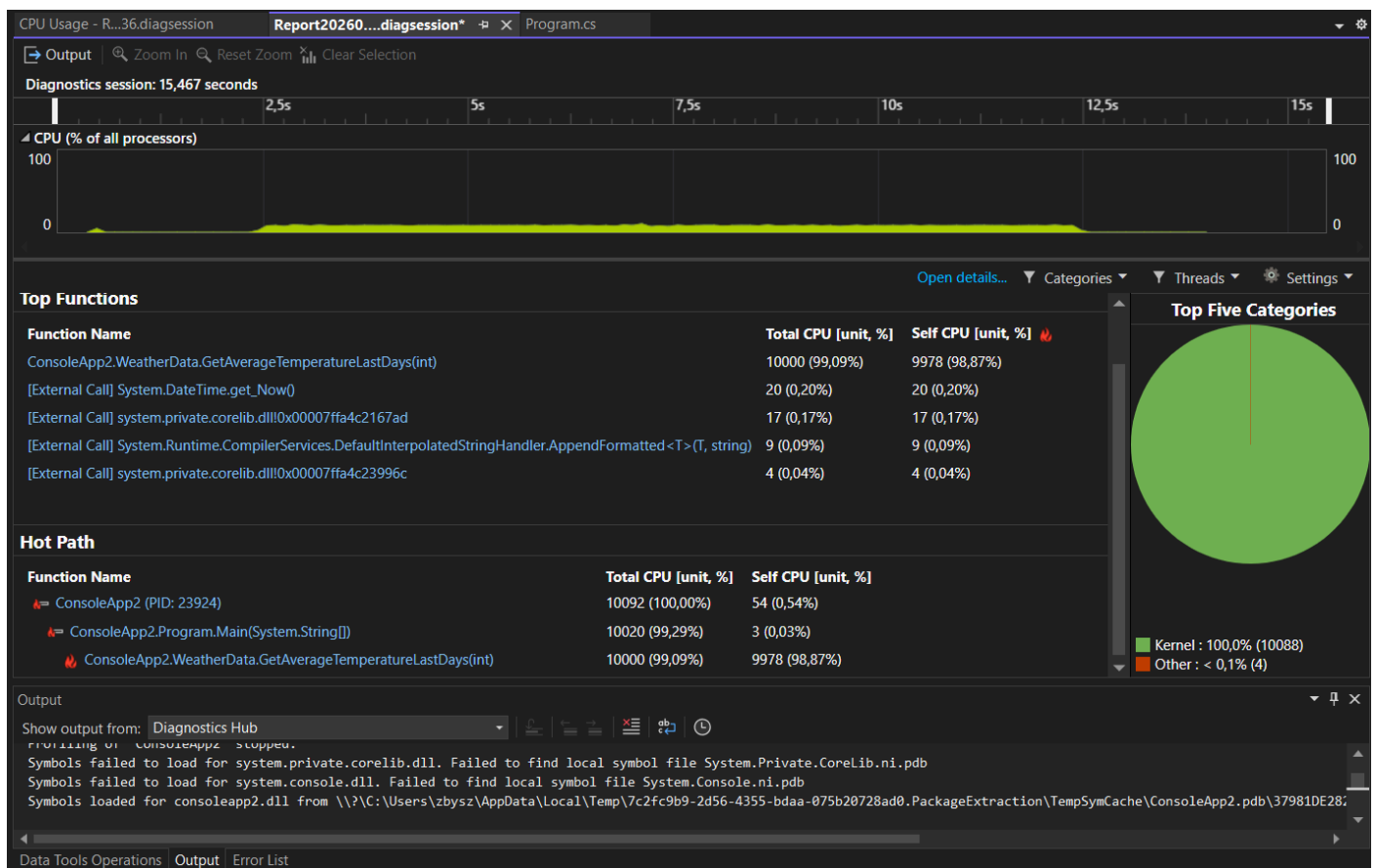
}
if (count == 0)
{
    return 0.0f;
}

```

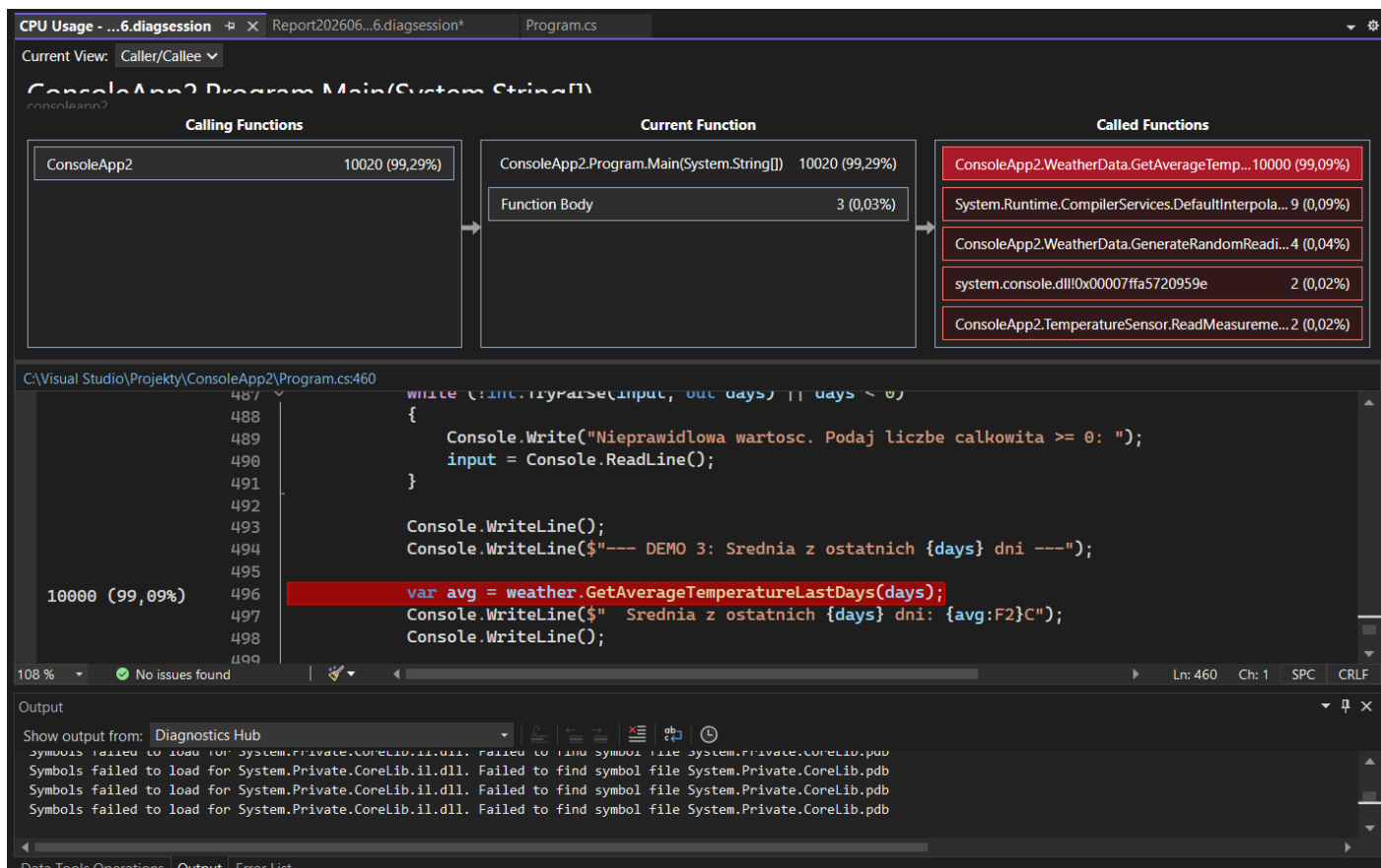
Wraz z aktualizacją pliku, błąd został oznaczony jako rozwiązany.

2. Użycie profilera do wykrycia hot spotów:

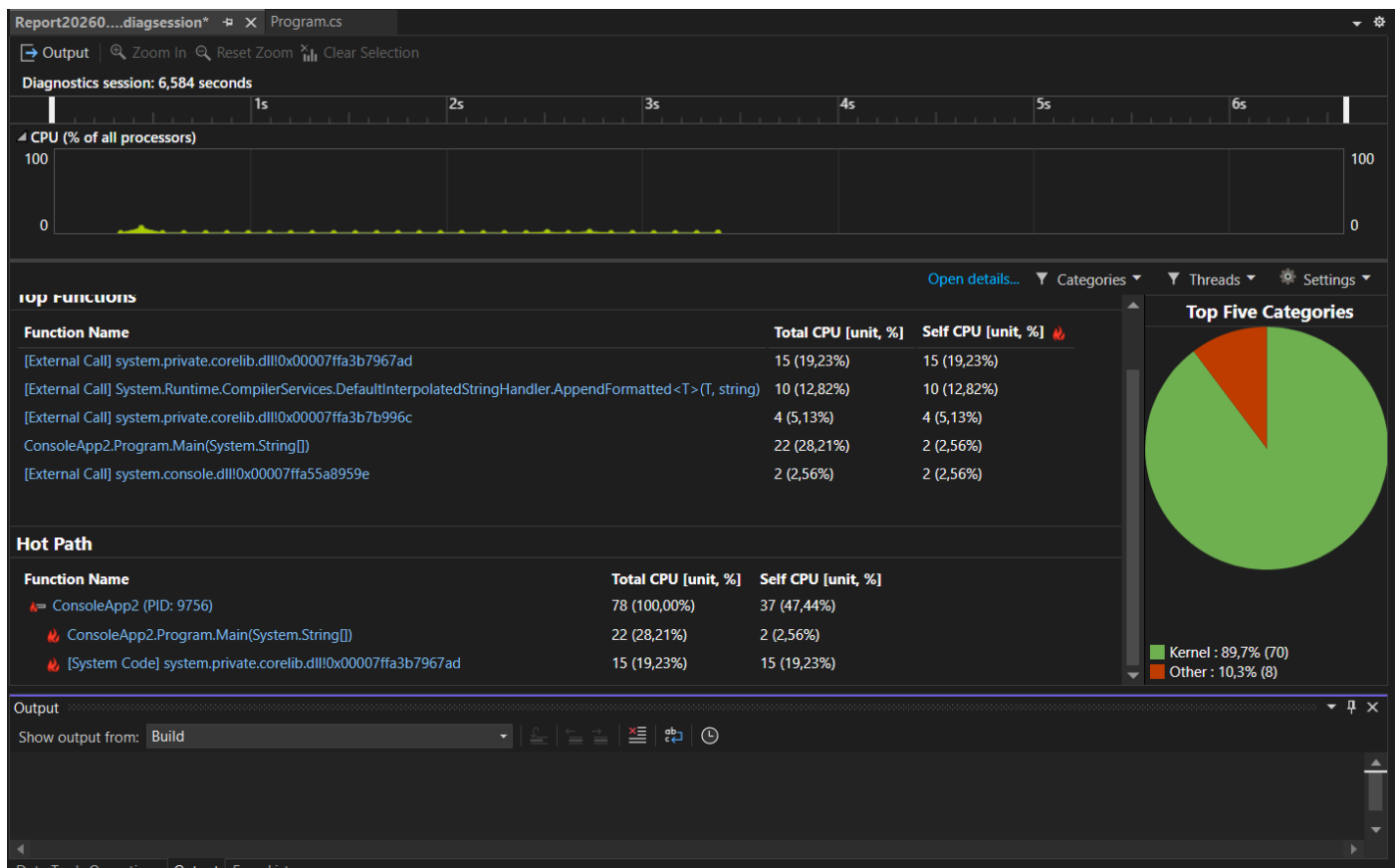
Użyto profilera wbudowanego w Visual Studio Code 2022. Wynik jego działania dla pierwotnej wersji programu prezentował się następująco:



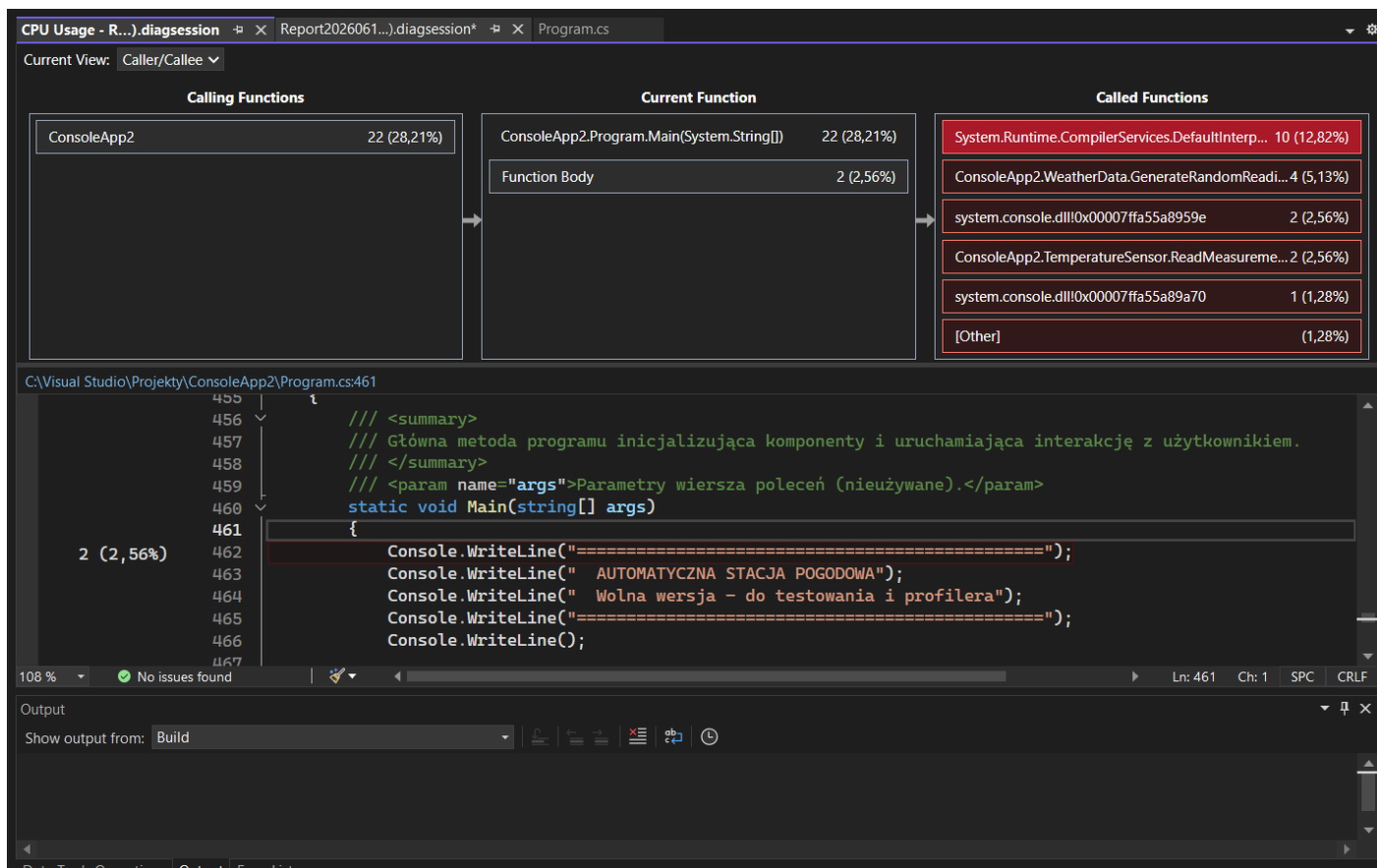
Wraz z widokiem szczegółowym:



Funkcja obliczająca średnią temperaturę z X ostatnich dni odpowiadała za zużycie około 99% zasobów procesora w czasie wykonywania programu. Przeprowadzono więc optymalizację oprogramowania. Po jej dokonaniu profiler zwrócił następujące wyniki:



Wraz z widokiem szczegółowym:



Jak można zauważyć hot spot w postaci funkcji `GetAverageTemperatureLastDays` zniknął. Optymalizacja pozwoliła na skrócenie czasu wykonywania programu o około 9 sekund (z 12,5 i do 3,5).